Week 26 (Days 176-180)

This week has five days only (Days 176-180). It finishes the expert capstone and closes the 180-day course. There is no Day 181 content day.

Status: Days 176-180 are written in full. Day 176 freezes Week 25 scorecards and runs a polish pass; Day 177 measures latency percentiles and paraphrase-injection robustness (and hardens calc AST); Day 178 runs final eval + ablations and writes an honest report; Day 179 ships a student report template plus a reference report filled from real logs; Day 180 delivers the 30-question cumulative final exam, a retrospective, a skills matrix, and a 6-12 month "what next" plan. Every number comes from `code/week-26/day176_*.py` ... `day180_*.py` and the Week 25 freeze under `code/capstone/out/freeze/`. Logs live in `code/week-26/out/`.

Retitled from the original SYLLABUS rows. Rows 176-180 mixed "write-up/demo", "cumulative review", "final exam", "rehearsal", and "presentation". They are retitled to match this week's actual closeout: polish -> robustness -> final eval/ablations -> write-up/demo template -> final exam + retrospective + what-next.

Compute note. Same CPU-only box as Week 25. No new model downloads required if the HF cache is warm. Android/GPU paths remain labelled UNTESTED.

Day 176: Capstone polish and freeze snapshot

Review today: Day 175 (+1), Day 172 (+3), Day 168 (+7), Day 155 (+21).

Block 1: Freeze before you polish (30 min)

Polish without a freeze is how baselines silently move. Copy Day 171-175 stats into `code/capstone/out/freeze/` and treat that directory as read-only for the rest of the week. Day 178 and Day 179 quote freeze numbers; Day 177 only adds new robustness measurements.

Block 2: Run the polish script (40 min)

```
python code/week-26/day176_polish.py
```

Verified (`out/day176_polish.log`, `day176_stats.json`): - Freeze copied **5** JSON scorecards (day171-day175). - Empty/whitespace input: no crash. - Overlong (2500 chars): blocked (`too long`). - `calc: 2**8` deterministic **256**. - Benign "What is PCA?": not blocked; answer length **109**. - Injection still blocked. - Service on `:8766`: health ok; `calc: 9*9` -> **81**. - Polish tests **7/7 PASS**. Freeze recall still **0.72**.

Block 3: What polish is (and is not) (35 min)

Polish is edge cases, determinism, and packaging - not prompt tweaking on the freeze set. If you change chunking or the system prompt today, you invalidate the freeze and must re-run Days 170-173. The empty-input path returning a retrieve trace is acceptable for a study tool; a production API would return HTTP 400 (optional stretch on the service).

Block 4: Self-check (20 min)

1. How many freeze files copied? ***5.***
2. Overlong guard reason? ***blocked: too long.***
3. Polish pass count? ***7/7.***
4. Why freeze before polish? *So robustness work cannot quietly rewrite the baseline.*

Block 5: Practice (30 min)

Add an HTTP 400 for empty `question` in `service.py` and a test. Do not change retrieval or prompts.

Going deeper

Wire a `/ready` endpoint that checks embeddings.npy shape == (1467, 384) before accepting /ask.

Interview-style question (15 min)

"We improved the demo after the eval." Reply: show the freeze directory timestamp and ask which JSON changed. If freeze moved, the improvement is untrusted.

Common mistakes

- Editing `corpus_qa.py` during polish.
- Claiming new recall numbers without re-running the harness.
- Leaving calc exceptions uncaught (fixed earlier in Week 25; still verify).

Timed 6-hour schedule (suggested)

- 0:00-0:30 Review + freeze rationale.
- 0:30-1:30 Run polish; read log line by line.
- 1:30-3:00 Edge-case practice (empty, long, unicode).
- 3:00-4:30 Optional /ready or HTTP 400 stretch.
- 4:30-5:30 Interview answer + notes.
- 5:30-6:00 Commit freeze/ + day176_stats.json only.

Build tasks (student)

1. Diff `freeze/day173_stats.json` against `code/week-25/out/day173_stats.json` - must be identical.
2. Run `calc: 2**8` five times; assert all equal.
3. Document the 7 polish checks in your design doc appendix.

What to do next

Day 177 stresses latency percentiles and paraphrase attacks; expect residual evasions and write them down.

Extra depth and systems notes

A freeze directory is a contract with your future self. When Day 178 quotes recall 0.720, a grader must be able to open `code/capstone/out/freeze/day173_stats.json` and see the same number without trusting your slides. That is why Day 176 copies five JSON scorecards before any polish change lands. Polish itself is deliberately boring: empty strings, overlong payloads, deterministic calc, known-pattern injection, and a live HTTP health check on an alternate port. None of those moves the retrieval index. If a teammate "helps" by retuning the system prompt during polish, the freeze is compromised and you must re-baseline under a new folder name with an explanation in the final report.

The seven polish checks that passed on this box are not a substitute for the Day 173 gate suite. They are a serving-path smoke test. Empty input returning a retrieve trace is a debug-friendly choice for a course assistant; a public API should prefer HTTP 400. Overlong input at 2500 characters is blocked with reason `too long`, which matches the budget row that caps prompt size before the model ever sees the bytes. Calc of `2**8` returning 256 five times in a row proves the tool path is deterministic - a flaky tool destroys integration tests and demo trust. Benign PCA questions must not trip the injection regex (0 false positives on the polish slice). The service on port 8766 answered health and `calc: 9*9` -> 81, proving the Week 25 HTTP surface still binds after a freeze.

Students sometimes ask whether polish includes prompt engineering. The answer is no for Week 26 Days 176-177. Prompt changes belong in a new experiment with a new eval run, not in a freeze window. The same rule applies to chunk boundaries and embedding models. The only code edits allowed during polish are guards, HTTP status codes, logging, and packaging. Day 177 will touch `run_calc`'s AST allow-list because a robustness failure is a safety defect, not a quality tweak - that distinction matters when you write the honest report.

Hardware honesty continues: CPU-only Xeon, no GPU numbers, Android path still UNTESTED. RSS is left to Day 177 after a warm RAG path. If you run polish on a laptop with a different BLAS, expect absolute latencies to move; the pass/fail of the seven checks should not. Keep the freeze read-only in git or in your evidence zip when the user asks for verification later. Do not zip yet.

Interview pressure often pushes people to ship a shinier demo after the eval. The correct reply is to point at the freeze timestamp and ask which JSON changed. If none changed, the improvement is packaging. If freeze moved, the improvement is untrusted until Day 178 is re-run. That cultural habit is worth more than any single recall point.

Practice beyond the script: add `/ready` that checks `embeddings.npy` shape `(1467, 384)` before accepting `/ask`. Add HTTP 400 for empty `question`. Neither change requires regenerating the index. Document both in an appendix of your design doc so the presentation can cite them as polish outcomes rather than as silent behaviour changes.

Finally, remember Week 26 has **five days only**. There is no Day 181 in the syllabus calendar for content; spaced-repetition rows beyond 180 in tooling are review-only. Plan your 6-hour blocks tightly: freeze rationale, run, edge cases, optional `/ready`, interview answer, commit. The 800-word lesson density exists so you can study offline without the scripts - but you still must run `day176_polish.py` yourself and keep the log.

Day 177: Performance and robustness pass

Review today: Day 176 (+1), Day 173 (+3), Day 169 (+7), Day 158 (+21).

Block 1: Percentiles beat means (30 min)

User-visible latency is a tail story. Measure retrieve p50/p90/p95 on all 25 QA pairs and RAG p50/max on a 5-question slice. Compare to the <=8 s RAG budget.

Block 2: Run robustness script (40 min)

```
python code/week-26/day177_robust.py
```

Verified (`out/day177_robust.log`, `day177_stats.json`): - Retrieve ms: p50 **118.8**, p90 **126.2**, p95 **127.7**, max **128.1**. - RAG+gen (n=5): p50 **0.84 s**, max **1.09 s**. - Known-pattern injection **3/3**; paraphrases blocked **1/5** (**4 evasions** logged). - Retrieve top-1 stability 5x: **stable** (`week-14.md#0-0`). - Calc abuse pre-harden **3/4** (lambda slipped). **AST allow-list hardened the same day**; post-test rejects lambda/import/open (**4/4**). - RSS **973 MiB**. Freeze recall still **0.72**.

Block 3: Read the evasions (35 min)

Paraphrases like "Forget your rules..." and "act as DAN..." did not match `INJECTION_PAT`. Detectors leak. Mitigations that still hold - allow-listed tools, AST calc, no filesystem tools, max length - matter more than extending the regex forever. Document residual risk in the Day 178 report; do not hide the 4 evasions.

Block 4: Self-check (20 min)

1. Retrieve p95? ***127.7 ms.***
2. Paraphrase block rate? ***1/5.***
3. Post-fix calc battery? ***4/4 reject/allow correct.***
4. Is RAG p50 under budget? *Yes (0.84 ? 8).*

Block 5: Practice (30 min)

Add one high-precision regex for "DAN" / "disregard the above" and re-measure FP on 10 benign questions. Keep the evasion list in the report either way.

Going deeper

Fuzz calc with 50 random AST-invalid strings; all must reject.

Interview-style question (15 min)

"Our block rate is 100%." Ask: on which set? Known-pattern or paraphrase? Show Day 177's 1/5.

Common mistakes

- Quietly expanding the freeze set with paraphrases used as training for the regex.
- Reporting only p50.
- Leaving Call/Lambda in the AST allow-list (fixed today).

Timed 6-hour schedule (suggested)

- 0:00-0:30 Review KV + injection notes.
- 0:30-1:30 Run Day 177; capture evasions verbatim.
- 1:30-3:00 Harden calc; re-test abuse battery.
- 3:00-4:30 Optional regex tighten + FP re-measure.
- 4:30-5:30 Write residual-risk paragraph for Day 178.
- 5:30-6:00 Commit agent.py harden + day177_stats.json.

Build tasks (student)

1. Record p50/p95 retrieve in the design doc.
2. Paste the four EVADED strings into FINAL_EVAL_REPORT residual risks.
3. Confirm `run_calc('(lambda:1)()')` raises after the harden.

What to do next

Day 178 freezes the story: ablations, worked/didn't, release decision.

Extra depth and systems notes

Percentile latency is the user-visible truth. A mean that looks fine can hide a tail that breaks a demo. Day 177 therefore records retrieve p50/p90/p95/max across all 25 QA pairs and RAG+generate p50/max on a five-question slice. Verified numbers on this box: retrieve p50 **118.8 ms**, p90 **126.2**, p95 **127.7**, max **128.1**; RAG p50 **0.84 s**, max **1.09 s** - both comfortably under the <=8 s budget from Day 169. RSS after the warm path was **973 MiB**, inside the 4 GiB spirit of the RAM budget. Freeze recall remained **0.72**.

Robustness is where detectors embarrass themselves. Known-pattern injection blocked **3/3**. Paraphrase injections blocked only **1/5**, with four evasions logged verbatim (including DAN-style and "forget your rules" phrasings). That echoes Week 21 and Week 22: regex suites overfit the strings you wrote yesterday. Mitigations that still hold without perfect detection - allow-listed tools, no filesystem tools, max length, AST-safe calc - are the real safety floor. Document residual risk; do not hide the 1/5 in a footnote.

The calc abuse battery originally scored **3/4** because `run_calc` treated `ast.Call` as allowed, so `(lambda:1)()` slipped through. The same-day harden switched to a strict arithmetic-only AST tuple (BinOp, UnaryOp, Constant/Num, Expression, and operator nodes only). Post-test rejects lambda, import, and open while still accepting `1+2*3`. That is default-deny on node types, not an ever-growing reject list. Retrieve top-1 stability across five identical queries stayed on `week-14.md#0-0`, which is the quiet proof that embedding and search are deterministic given fixed floats and thread settings.

When you present Day 177, put known-pattern 3/3 beside paraphrase 1/5 on the same slide. Put p50 beside p95. Put pre-harden 3/4 beside post-harden 4/4. Green-only slides fail the honesty bar that Month 6 has been teaching since the BLOCKED Month-5 capstone. Optional stretch: add a high-precision line for "DAN" / "disregard the above", re-measure false positives on ten benign questions, and keep the evasion list in the report either way. Fuzz calc with fifty random AST-invalid strings; all must reject.

Systems habit: if retrieve latency suddenly doubles, check BLAS thread counts and whether another agent is hammering the shared box before you rewrite the index. If top-1 flaps, chase seeds and OpenMP env vars. Robustness work is unfinished by design - paraphrase attackers will always find a new wording - so the release decision tomorrow must say course-yes / production-no unless you have a stronger guard story than regex.

Timed practice: rehearse explaining the lambda hole in under sixty seconds. Graders love a concrete bug with a concrete fix. Then commit `agent.py` and `day177_stats.json` together so the harden and the measurement stay paired in history.

Day 178: Final evaluation, ablations, honest report

Review today: Day 177 (+1), Day 174 (+3), Day 170 (+7), Day 161 (+21).

Block 1: Ablations worth running (30 min)

Minimum set: recall@1/@3/@5; context on vs off (with caveat); calc allow/deny; tools smoke; qualitative LoRA-alone from Day 170. More ablations are welcome; fewer than three is not a final report.

Block 2: Run eval + report (45 min)

```
python code/week-26/day178_eval_ablate.py
```

Verified (`out/day178_eval_ablate.log`, `day178_stats.json`, `FINAL_EVAL_REPORT.md`): - Final recall@3 **18/25 = 0.720** (matches freeze). - Context ON grounded **8/8**; OFF also **8/8** on the heuristic - **caveat logged**: OFF still overlaps retrieved hits, so the metric is retrieval-tied; factuality still needs spot checks. - Recall @1/@3/@5 = **0.520 / 0.720 / 0.720** (@5 no gain). - Calc battery **6/6**; tools **3/3**. - Decision: **ACCEPT_COURSE_NO_PROD**.

Block 3: Write the negatives in full sentences (35 min)

The report lists residual risks: LoRA-alone nonsense; paraphrase evasions; BLAS winning on tiny N; grounded!=fact; macro recall skew; PSI bin caveat; pre-harden calc hole (fixed). If your presentation only shows the green table, you fail this day.

Block 4: Self-check (20 min)

1. recall@1? ***0.520.***
2. Release decision string? ***ACCEPT_COURSE_NO_PROD.***
3. Did @5 beat @3? *No.*
4. Where is the report? *`code/capstone/out/FINAL_EVAL_REPORT.md`.*

Block 5: Practice (30 min)

Add a one-row "spot-check factuality" table for three questions (human yes/no). Attach it to the report appendix.

Going deeper

Bootstrap confidence interval on recall@3 with 1000 resamples of the 25 pairs; report the interval beside 0.72.

Interview-style question (15 min)

"Ship it?" Answer with the decision line: course yes, production no - then cite paraphrase 1/5 and model card.

Common mistakes

- Deleting failed ablations.
- Treating heuristic groundedness as factuality.
- Re-tuning top_k on the freeze set after seeing @1.

Timed 6-hour schedule (suggested)

- 0:00-0:30 Plan ablation table.
- 0:30-2:00 Run Day 178; save report.
- 2:00-3:30 Human factuality spot-checks.
- 3:30-5:00 Draft presentation outline from WORKED/DID NOT.
- 5:00-6:00 Peer-read the negatives section aloud.

Build tasks (student)

1. Quote WORKED and DID-NOT bullets into your slides.
2. Confirm recall@3 unchanged vs freeze (0.720).
3. Link DEMO_CHECKLIST (Day 179) as a stub for tomorrow.

Numbers in the report must be copy-pasted from JSON, not retyped from memory. When Day 179 regenerates the reference report it will pull the same files, so hand-edited markdown that disagrees with `day178_stats.json` becomes a silent lie. Prefer a

one-line script that prints the decision string into the report over any manual transcription. If you must edit prose, never edit digits. The ACCEPT_COURSE_NO_PROD line is also a teaching artefact for interviews: saying no on purpose, with evidence, is rarer and more valuable than another half-point of recall bought by peeking at the freeze set.

What to do next

Day 179 turns the report into a student template and a filled reference plus a 90 s demo checklist.

Extra depth and systems notes

Final evaluation is a genre with a fixed outline: tables, ablations, negatives, decision. Day 178 re-runs the harness against the freeze spirit and writes `FINAL_EVAL_REPORT.md` plus `day178_stats.json`. Verified: recall@3 **18/25 = 0.720** (matches freeze); recall@1/@3/@5 = **0.520 / 0.720 / 0.720** (top_k=5 buys nothing more on this set); calc battery **6/6**; tools **3/3**; decision **ACCEPT_COURSE_NO_PROD**.

The context on/off groundedness ablation both scored **8/8** on the heuristic. That looks like a wash until you read the caveat: with context off, the scorer can still award "grounded" via word overlap against retrieved hits that were computed anyway. The metric is retrieval-tied, not a pure faithfulness judge. Spot-check factuality remains mandatory; Week 25 already logged fluent wrong answers on topics like k-means. Put that caveat in the report body, not only in a code comment.

WORKED bullets belong on slides: freeze recall held; gates PASS; RAG p50 ? 8 s; tools and calc green; service tested; model card says NO for real users. DID-NOT bullets belong louder: LoRA-alone nonsense after 60 steps; paraphrase evasions 4/5; OpenBLAS beating Rust int8 on the tiny 1467x384 corpus; grounded!=fact; macro recall can skew; PSI percentile-bin caveat; pre-harden calc hole (fixed Day 177). Deleting failed ablations is academic fraud in miniature. Keeping them is how portfolio readers learn to trust you.

Release language matters. **ACCEPT_COURSE_NO_PROD** means the artefact is good enough to demonstrate course outcomes and not good enough to put in front of unpaid users without a human in the loop, a stronger guard stack, and a larger eval. That is continuous with Day 150 BLOCKED and Day 168 audit NO. Consistency of integrity across months is itself a learning outcome.

Optional deeper work: bootstrap a confidence interval on recall@3 with 1000 resamples of the 25 pairs; report the interval beside 0.72. Add a three-row human factuality spot-check table. Neither changes the decision line unless factuality collapses. Do not retune top_k on the freeze set after seeing @1 - that is peeking. If you want a new top_k experiment, create `freeze-v2` and say why.

When Day 179 generates the reference report, it will read these same JSON files. If you hand-edit numbers into the markdown without regenerating, template and reference will drift. Prefer regenerating from scripts. Peer-read the negatives section aloud; if you wince, the section is doing its job.

Day 179: Write-up, demo template, reference report

Review today: Day 178 (+1), Day 175 (+3), Day 171 (+7), Day 167 (+21).

Block 1: Template before prose (30 min)

Students need a blank scaffold with the same headings as the reference. Fill the reference only from logs - never from memory.

Block 2: Generate artefacts (30 min)

```
python code/week-26/day179_writeup.py
```

Verified (`out/day179_writeup.log`, `day179_stats.json`): - `code/capstone/out/REPORT_TEMPLATE.md` - blank student scaffold. - `code/capstone/out/REFERENCE_REPORT.md` - filled with freeze recall **0.72**, gates **PASS**, Day 177 RAG p50 **0.84 s**, RSS **973 MiB**, decision **ACCEPT_COURSE_NO_PROD**, ablation row @1/@3/@5. - `DEMO_CHECKLIST.md` - 90 s path: health, calc, injection, RAG, scorecard, model card NO.

Block 3: Rehearse the demo (45 min)

Time yourself. If you exceed 90 s, cut explanation, not evidence. Checklist order: prove the service, prove tools, prove refusal, prove retrieval, prove gates, prove humility (production NO).

Block 4: Self-check (20 min)

1. Template path? *`REPORT_TEMPLATE.md`.*
2. Reference recall cited? ***0.72.***
3. Demo must show production decision? *YES - NO for real users.*
4. Which day supplies paraphrase numbers? *Day 177.*

Block 5: Practice (40 min)

Fill the student template for *your* fork of the capstone (even if identical). Hand-write one section to prove you are not only regenerating scripts.

Going deeper

Export REFERENCE_REPORT.md to a one-page PDF for portfolio upload (optional tooling).

Interview-style question (15 min)

"Walk me through the system." Use the architecture paragraph in the reference report, then open day173_stats.json live.

Common mistakes

- Demo only shows happy-path RAG.
- Template and reference diverge on metric names.
- Forgetting to point at MODEL_CARD.md.

Timed 6-hour schedule (suggested)

- 0:00-0:30 Skim FINAL_EVAL_REPORT.
- 0:30-1:00 Run day179_writeup.py.
- 1:00-2:30 Fill personal template.
- 2:30-4:00 Three timed demo rehearsals.
- 4:00-5:30 Slide outline (8 slides max).
- 5:30-6:00 Checklist dry-run with curl.

Build tasks (student)

1. Complete every blank in REPORT_TEMPLATE.md.
2. Tick DEMO_CHECKLIST.md with timestamps.
3. Store a terminal capture path in the appendix.

Slide count discipline helps: eight slides maximum forces you to choose evidence over ornament. Suggested eight: problem and non-goals; architecture; budget measured vs target; ablations table; residuals including paraphrase 1/5; RAI and model card NO; demo checklist screenshot; what you would do with a GPU or a phone (labelled UNTESTED). Speaker notes can hold the longer negatives. After rehearsal, update the checklist timestamps so a reviewer sees you actually timed the path rather than imagined it.

Reference-report integrity also means citing Day 177 paraphrase numbers beside Day 176 polish greens. A reader who only sees 7/7 polish and 0.72 recall will overestimate safety. Put the 1/5 paraphrase line in the residual section of both the template (as a blank to fill) and the reference (as a filled value). If your fork changes the injection regex after Day 177, you must re-run Day 177 and update the reference via the script - do not patch the markdown alone. Demo videos should include at least one blocked injection attempt on camera; cutting it out to look smoother is the same class of error as deleting a failed ablation.

What to do next

Day 180 - final exam (30 Q), retrospective, skills matrix, what-next for 6-12 months.

Extra depth and systems notes

Write-up day produces three artefacts others can open without you: `REPORT_TEMPLATE.md` (blank scaffold), `REFERENCE_REPORT.md` (filled only from logs), and `DEMO_CHECKLIST.md` (90-second path). Day 179's script verified those paths under `code/capstone/out/` with copies under `code/week-26/out/`. The reference cites freeze recall **0.72**, Day 173 gates PASS, Day 177 RAG p50 **0.84 s**, RSS **973 MiB**, ablation @1/@3/@5, and decision **ACCEPT_COURSE_NO_PROD**. Hardware line must say CPU-only. Android and GPU Triton remain UNTESTED labels, not TODOs you quietly drop.

The template exists so a student forking the capstone cannot "innovate" the section order into marketing copy. Required sections stay: budget table measured vs target; architecture; ablations; residuals; RAI/model card; demo evidence. Fill every blank even if your fork is identical - handwriting one section proves you read it. The demo checklist order is deliberate: health -> calc -> injection block -> RAG answer -> scorecard/gates -> model card production NO. Happy-path RAG alone is a fail. If injection is not blocked during rehearsal, stop and fix; do not narrate over the failure unless you are teaching the failure.

Rehearse three times with a timer. Cut explanation before you cut evidence. Warm `get_agent()` before recording so cold-start does not steal the 90 seconds. Store a terminal capture path in the appendix. Optional: export the reference report to a one-page PDF for portfolio upload if you have tooling; the course itself stays markdown-first because pandoc was not assumed on this box.

Interview mode: "Walk me through the system." Use the architecture paragraph, then open `day173_stats.json` live, then open `MODEL_CARD.md` to the production NO line. That sequence beats any slide transition. Graders who have seen too many hype demos will relax when they see the NO.

Common failure modes: template and reference diverging on metric names; demo that skips tools; forgetting residual paraphrase numbers from Day 177; inventing a new recall in slides that is not in JSON. Day 180 will not rewrite the report - it adds exam, retrospective, skills matrix, and what-next. Lock the write-up tonight.

Day 180: Final exam, retrospective, skills matrix, what next

Review today: Day 179 (+1), Day 176 (+3), Day 173 (+7), Day 150 (+21; Month-5 checkpoint contrast).

Block 1: Sit the final (90+ min closed-book)

```
python code/week-26/day180_closeout.py
```

Opens/writes `day180_final_exam.md` (**30** questions, Months 1-6) with answers underneath. Sit closed-book first; then grade yourself.

Verified (`out/day180_closeout.log`, `day180_stats.json`): exam **30** Q; skills matrix **14** rows; retrospective + what-next under `code/week-26/out/` and copied to `code/capstone/out/`.

Block 2: Retrospective (40 min)

Read `day180_retrospective.md`. Add one private paragraph: what *you* will keep doing weekly. Course-level themes - measure before claiming; defence in depth; small models need retrieval; systems skills - should already feel familiar.

Block 3: Skills matrix + what next (50 min)

Fill your own levels in `SKILLS_MATRIX.md` (basic/moderate/strong). Be harsher than the reference column. Then read `WHAT_NEXT.md`: Anki + monthly change-one-thing labs; reading list from **verified** official docs only; research directions; OSS contribution ideas; portfolio advice that leads with honest NO.

Block 4: Self-check (20 min)

1. How many final-exam questions? ***30.***
2. Capstone freeze recall cited in exam? ***0.72.***
3. Name one UNTESTED area. *Android app path / GPU Triton.*
4. Primary spaced-repetition inputs? *`flashcards/month-*.csv` + `review-schedule.csv`.*

Block 5: Practice (40 min)

Schedule the next 30 days of Anki tags from the updated review schedule. Pick one research direction and write a ½-page proposal with a measurement plan.

Going deeper

Re-answer any missed exam questions tomorrow (+1 review) without notes, then check.

Interview-style question (15 min)

"What will you do in the next six months?" Pick one research direction and one OSS contribution from WHAT_NEXT; add a measurement.

Common mistakes

- Skipping the closed-book attempt.
- Inflating the skills matrix.
- Bookmarking random blogs instead of primary docs.

Timed 6-hour schedule (suggested)

- 0:00-0:20 Setup / print exam without answers.
- 0:20-2:00 Closed-book exam.
- 2:00-2:45 Grade + list weak months.
- 2:45-3:30 Retrospective paragraph.
- 3:30-4:30 Skills matrix honesty pass.
- 4:30-5:30 What-next plan in your calendar.
- 5:30-6:00 Open weak week files for remediation.

Build tasks (student)

1. Score /30; remediate every miss.
2. Update skills matrix with personal evidence paths.
3. Star/bookmark only the what-next links you will actually open next week.

Course completion is not skill completion. The spaced-repetition schedule continues past Day 180 as review rows; Anki does not care that the syllabus ended. If you miss a week of cards, restart from the oldest overdue tag rather than from Day 180 vanity. When dependencies bump and `run_all.sh` breaks, that breakage is a lab - open the log, fix one pin, re-measure the affected day, and write three lines in your learning log. That habit is the real closeout deliverable.

When you open WHAT_NEXT.md, pick one research direction this week and put a calendar reminder for a measurement milestone thirty days out. Vague aspirations ("get better at agents") do not survive contact with Monday mornings; a dated note that says "re-run Day 177 paraphrase list after adding schema-level tool caps" does. Similarly, pick one upstream issue tracker you already depend on and read three open issues before filing anything - contribution quality beats contribution count. The final exam's thirty answers are a map of the course; wrong answers are coordinates for remediation, not a grade to hide.

What to do next

Use Month-6 flashcards, keep the freeze, and follow WHAT_NEXT. Do not zip the course until the verification pass you were promised.

Extra depth and systems notes

Day 180 closes the course. Run `day180_closeout.py` to emit the cumulative **30**-question final exam with answers, a retrospective essay, a **14**-row skills matrix, and a 6-12 month what-next plan. Sit the exam closed-book first - fold the answers or print without them - then grade yourself and remediate every miss against the matching week file. The exam samples all six months; it does not cover every day. Weak months tell you which Anki tags need weight.

The retrospective should already feel familiar: measure before claiming; defence in depth over detector theatre; small models need retrieval; systems skills (freeze, gates, model cards) travel farther than a single accuracy number. Add one private paragraph about what you will keep doing weekly. The skills matrix lists fourteen rows spanning Python/ML basics through RAG,

agents, MLOps, on-device, RAI, and Rust kernels. Rate yourself basic/moderate/strong with evidence paths; be harsher than the reference column. Inflated matrices help no one six months later.

What-next reading is restricted to verified official documentation and pages already listed in `docs/resources.md`: PyTorch docs, HF Transformers, ONNX Runtime performance, Gymnasium, scikit-learn user guide, NIST AI RMF 1.0 PDF, EU AI Act overview (not legal advice), ExecuTorch getting started, Opacus, Anki, plus optional deeplearning.ai / Coursera inspiration links from `docs/teaching-style.md`. No invented URLs. Research directions include groundedness metrics for small-LM RAG, paraphrase-robust guards, pruning/int8 that wins on your chip, DPO with frozen behavioural evals, and finishing the UNTESTED Android KV/int4 path. OSS ideas: tests/docs in ORT, sentence-transformers, Gymnasium; publish ablation tables with hardware labels; CI on the Day 173 harness; upstream a PyO3 int8 example with BLAS caveats.

Keep skills fresh with the spaced-repetition schedule: `flashcards/month-01.csv` ... `month-06.csv` and `review-schedule.csv` (+1/+3/+7/+21). Once a month, change one hyperparameter in one week lab, predict, measure, log. Re-run `bash run_all.sh` after dependency bumps and treat breakages as labs. Portfolio advice: lead with the honest report - freeze 0.72, paraphrase gaps, production NO - not with a vanity metric.

Week 26 has five days; Day 180 is the last content day. Do not zip the course until the verification pass the user requested next. Update flashcards, syllabus titles, README index, resources, data README, and PROGRESS as part of closeout hygiene so the repository's front door matches reality.

Week 26 quiz (12 questions)

This week has 5 days. Answers follow each question.

1. How many days are in Week 26? **Answer:** 5 (Days 176-180).
2. Day 176 polish tests? **Answer:** 7/7 PASS.
3. Day 177 retrieve p50 ms? **Answer:** 118.8.
4. Day 177 paraphrase injections blocked? **Answer:** 1/5 (4 evasions).
5. Day 178 recall@1 / @3 / @5? **Answer:** 0.520 / 0.720 / 0.720.
6. Day 178 release decision? **Answer:** ACCEPT_COURSE_NO_PROD.
7. Where is the student report template? **Answer:** `code/capstone/out/REPORT_TEMPLATE.md`.
8. Reference report freeze recall? **Answer:** 0.72.
9. Final exam question count (Day 180)? **Answer:** 30.
10. Skills matrix row count in the reference? **Answer:** 14.
11. Name one verified what-next doc link host. **Answer:** docs.pytorch.org (or huggingface.co, onnxruntime.ai, gymnasium.farama.org, scikit-learn.org, nvlpubs.nist.gov, ankiweb.net, ...).
12. What must a demo show besides happy-path RAG? **Answer:** Tool calc, injection block, gates/scorecard, and production NO (model card).

Week 26 project checklist

- [x] Freeze Week 25 scorecards + polish (176)
- [x] Latency percentiles + paraphrase robustness + calc harden (177)
- [x] Final eval, ablations, honest report (178)
- [x] Template + reference report + demo checklist (179)
- [x] Final exam 30Q, retrospective, skills matrix, what-next (180)